

HammerTime

A Dynamic Spatio-Temporal Graph Neural Network
for Formula 1 Traffic Delay Estimation

Project Report (50% Core Implementation)

Case study: 2021 Abu Dhabi Grand Prix

A. Adithya Sai Pratheek

24BAI1445

SCOPE – VIT Chennai

M.S. Abhinav Sriram

24BAI1209

SCOPE – VIT Chennai

October 6, 2026

Abstract

In Formula 1, a car that follows closely behind another loses downforce and tyre performance because it drives in the leader’s turbulent wake (“dirty air”). This loss is rarely measured directly. HammerTime is a project that estimates it as a per-car, per-mini-sector time delay Δt_{sector} with a dynamic spatio-temporal graph neural network (ST-GNN). This report describes the work completed at the half-way point.

The 50% core implementation ingests real telemetry of the 2021 Abu Dhabi Grand Prix through the FastF1 library (416,879 aligned rows, 19 drivers), resamples it to a 4 Hz grid, computes the time gap between consecutive cars, and builds 21,941 graph snapshots with 137,355 directed leader-to-follower edges weighted by a Gaussian radial basis function (RBF). A lightweight message-passing network (3,393 trainable parameters) with temporal pooling is trained on 4,112 windows and evaluated on the final 1,371 windows of the race. The data pipeline, the graph builder, the graph visualiser, the training loop and the analytics module all run end to end.

The prediction accuracy does not yet meet the targets: on the held-out windows the model reaches an MAE of 1.14 s and an R^2 of 0.29, against targets of < 0.15 s and 0.68–0.82. This report gives these numbers without adjustment and analyses their causes. Code inspection shows that the target look-up used for training is not lap-aware, which means that the current test figures are a preliminary baseline and not a valid estimate of generalisation. The report also covers the Safety Car period of the race, target clipping, label quantisation and the single-hop design of the model, and it sets out a concrete, testable plan for the second half of the project.

Keywords: Formula 1, graph neural network, spatio-temporal learning, dirty air, FastF1, message passing, traffic delay, motorsport analytics.

Contents

1	Introduction	6
1.1	Background: Aerodynamics and Racing in Traffic	6
1.2	Why the 2021 Abu Dhabi Grand Prix	6
1.3	Motivation	6
1.4	Problem Statement and Formalisation	6
1.5	Objectives	7
1.6	Work Completed in the First Half	7
1.7	Organisation of this Report	7
1.8	Notation	8
2	Background and Related Work	8
2.1	Graph Neural Networks	8
2.2	Spatio-Temporal Graph Models	8
2.3	Position of this Work	9
2.4	Tools	9
3	Requirements and Design Rationale	9
3.1	Requirements	9
3.2	Design Decisions	10
3.3	Two Fixed Rules	10
4	Data Ingestion and Preprocessing	10
4.1	Source Data	10
4.2	Schema of the Aligned Table	11
4.3	Temporal Alignment	11
4.4	Mini-Sectors and Lap Counter	11
4.5	Gap to the Car Ahead	11
4.6	Clean-Air Targets	12
4.7	Data Quality Notes	12
5	Dynamic Graph Construction	13
5.1	Edge Rule and Weight	13
5.2	Kernel Width Sensitivity	14
5.3	Disturbance Tiers	14
5.4	Node Features	14
5.5	Construction Algorithm	15
5.6	Structural Property: The Graph is a Set of Chains	15
5.7	Graph Statistics	15
5.8	Worked Example	15
6	Model: CoreSTGNN	16
6.1	Architecture Overview	16
6.2	Spatial Message Passing	16
6.3	Temporal Pooling	17
6.4	Regression Head	17

6.5	Parameter Count	17
6.6	Computational Cost	17
6.7	Limits of the Core Design	17
7	Experimental Setup	18
7.1	Training Samples	18
7.2	Data Split	18
7.3	Hyper-parameters	19
7.4	Loss and Metrics	19
7.5	Reproducibility and Commands	19
7.6	Runtime	20
8	Results	20
8.1	Execution Log	20
8.2	Dynamic Graph Snapshot	21
8.2.1	Reading the Snapshot	21
8.3	DRS-Train Detection	22
8.4	Training Convergence	22
8.5	Test Performance	23
8.6	Traffic Analytics	23
8.6.1	Left Panel: Parity Plot	23
8.6.2	Right Panel: Label Distribution	23
8.7	Interpretation of the Train–Test Gap	24
9	Analysis, Limitations and Threats to Validity	24
9.1	Findings	24
9.2	Summary Table	25
9.3	What Works	25
10	Verification and Validation	26
10.1	Checks Performed	26
10.2	Checks Planned	26
11	Project Management	26
11.1	Status at the Half-Way Point	26
11.2	Risk Register	27
12	Plan for the Second Half	27
12.1	From the 50% Core to the 100% System	27
12.2	Planned Attention Layer	27
12.3	Experiment Plan	28
12.4	Success Criteria	28
12.5	Planned Visual Suite	28
13	Conclusion	28
A	Glossary	29

B Repository Layout	30
C Notes on Configuration	30
D Key Code Excerpts	30

List of Figures

1	Gaussian RBF edge weight for three kernel widths. The dashed lines mark the tier limits and the edge limit at 2 s.	13
2	Architecture of CoreSTGNN. The spatial part (projection and message passing) runs on every snapshot of the window. The results are pooled over time and mapped to one delay value per car.	16
3	Layout of one training sample. Each box is one 0.25 s snapshot.	18
4	Terminal output of <code>demo_core.py --mode full --epochs 3</code>	20
5	Dynamic disturbance graph at snapshot 3000 ($t = 750$ s).	21
6	Training loss of the two runs (only the epochs listed in the reports are plotted).	22
7	Left: predicted versus actual delay on the test windows (the dashed line is parity). Right: distribution of the actual delay in the test windows.	23

List of Tables

1	Notation used in this report.	8
2	Qualitative comparison of modelling choices for traffic delay.	9
3	Requirements of the core (50%) implementation.	9
4	Main design decisions and their rationale.	10
5	Columns of the aligned telemetry table.	11
6	Edge weight e_{ij} for selected gaps and kernel widths.	14
7	Disturbance tiers used for graph visualisation.	14
8	Output of the graph construction stage.	15
9	Layers and trainable parameters of CoreSTGNN.	17
10	Hyper-parameters of the core experiment.	19
11	Stage timing taken from the execution log (3-epoch run, cached data).	20
12	Active edges at snapshot 3000. Gaps are rounded to two decimals in the log.	21
13	Training loss (masked L1, seconds per sector).	22
14	Performance on the held-out closing windows (laps 44–58).	23
15	Issue register with evidence, expected effect and planned action.	25
16	Checks that were performed on the 50% implementation.	26
17	Status of project components at the half-way point.	26
18	Main risks and mitigations.	27
19	Planned upgrades from the 50% core to the 100% system.	27
20	Planned experiments.	28
21	Files of the 50% core implementation.	30

1 Introduction

1.1 Background: Aerodynamics and Racing in Traffic

A Formula 1 car produces a large part of its grip from aerodynamic downforce. The front wing, floor, diffuser and rear wing shape the air around the car, and the air that leaves the car is turbulent and slower than the free stream. A second car that drives close behind meets this disturbed air. The follower then produces less downforce, especially at the front axle. Corner speed falls, the car slides more, and the tyres run hotter. The effect is strongest in fast corners and decreases as the gap grows.

This effect has two consequences for race strategy. First, a faster car often cannot pass a slower one, because the follower loses the pace it needs to close up through the corners. Second, a car that stays in traffic wears its tyres faster than a car in clean air, so the lap time loss is not limited to the lap on which it is measured. The Drag Reduction System (DRS) lets a follower within one second of the car ahead open a rear-wing flap on designated straights, but it does not remove the loss in the corners. A group of cars that all run within DRS range of each other is called a *DRS train*. In such a train the cars move at the pace of the slowest car, and passing becomes very hard.

1.2 Why the 2021 Abu Dhabi Grand Prix

The 2021 Abu Dhabi Grand Prix is a convenient case study for three reasons. It is the last race of the season, so the data is complete and well documented. It has a full field with many close battles, including the lead fight between M. Verstappen (VER) and L. Hamilton (HAM). It also contains a Safety Car period from lap 53, after the crash of N. Latifi, which makes it a hard test case: the final laps of the race do not follow the normal racing physics that the model is meant to learn. The project uses this fact as a diagnostic (see Section 9).

1.3 Motivation

Teams know the dirty-air effect qualitatively, but broadcast graphics and public timing screens show only the gap in seconds. They do not show how much lap time the follower actually loses. A model that estimates this *traffic delay* for every car at every place on the circuit would help in three ways:

- it would explain *why* a faster car cannot pass, with a number;
- it would give strategists a cost for staying in traffic, which supports pit-stop decisions;
- it would give broadcasters a clear visual story (who is slowing whom).

1.4 Problem Statement and Formalisation

A race is a sequence of states of the whole field. Let $\mathcal{V} = \{1, \dots, N\}$ be the set of cars ($N = 19$ in the data used here) and let $t \in \{0, \Delta\tau, 2\Delta\tau, \dots\}$ be discrete time with $\Delta\tau = 0.25$ s. At each time step, the field is a directed graph

$$\mathcal{G}_t = (\mathcal{V}, \mathcal{E}_t, \mathbf{X}_t, \mathbf{e}_t), \quad (1)$$

where $\mathbf{X}_t \in \mathbb{R}^{N \times F}$ holds the node features, $\mathcal{E}_t$ is the set of directed leader-to-follower edges and $\mathbf{e}_t$ holds the edge weights. Given the last W graphs,

$$\hat{\mathbf{y}}_{t+h} = f_{\theta}(\mathcal{G}_{t-W+1}, \dots, \mathcal{G}_t), \quad \hat{\mathbf{y}} \in \mathbb{R}^N, \quad (2)$$

the task is to predict for every car i the extra time

$$\Delta t_{\text{sector}}^{(i)} = t_{\text{actual}}^{(i)} - t_{\text{clean-air}} \quad (3)$$

that the car needs for the mini-sector it will be in at time $t + h$, compared with the time that the same sector takes in clean air.

1.5 Objectives

1. Build a repeatable pipeline that turns raw race telemetry into an aligned time-series table that includes the gap to the car ahead.
2. Represent the race as a sequence of directed graphs in which an edge means “car j disturbs car i ”, with a physically motivated weight.
3. Train a spatio-temporal GNN to predict Δt_{sector} .
4. Visualise the disturbance graph and the traffic statistics in a form that is suitable for broadcast use.
5. (Second half) Add attention, safety-car filtering, baselines and a physics validation suite, and reach the accuracy targets.

1.6 Work Completed in the First Half

- A FastF1 ingestion and alignment module with a Parquet cache.
- A gap computation and clean-air target module.
- A dynamic graph builder (21,941 snapshots; 137,355 edges).
- A tiered, broadcast-style graph visualiser with an adjacency matrix.
- A trainable message-passing ST-GNN with a masked loss and a chronological train/test protocol.
- An analytics module (DRS-train detector, parity plot, histogram).
- A command-line runner with three modes (`full`, `graph`, `train`).
- A critical review of the results, with a list of confirmed and suspected problems and a plan to fix them.

1.7 Organisation of this Report

Section 2 reviews related work. Section 3 states the requirements and the main design decisions. Sections 4 to 6 describe the data, the graph and the model. Section 7 gives the experimental protocol, Section 8 the results, and Section 9 the analysis and limitations. Sections 10 to 12 cover verification, project management and the plan for the second half. Section 13 concludes.

1.8 Notation

Table 1: Notation used in this report.

Symbol	Meaning
$N, \mathcal{V}$	Number of cars (nodes) and the set of cars
$\Delta\tau$	Sampling interval (0.25 s at 4 Hz)
Δt_{ij}	Time gap (s) between leader j and follower i
e_{ij}	Edge weight (Gaussian RBF of the gap)
σ	Kernel width (1.0 s)
$\mathbf{x}_i \in \mathbb{R}^F$	Node feature vector ($F = 4$ in the core model)
$\mathbf{h}_i \in \mathbb{R}^D$	Hidden state of car i ($D = 32$)
W, K	Window length and prediction offset (both 5 snapshots)
S	Number of mini-sectors per lap (20)
Δt_{sector}	Target: sector time minus clean-air baseline
m_i	Mask: 1 if car i has a label in the sample, else 0

2 Background and Related Work

2.1 Graph Neural Networks

Graph neural networks learn a representation of each node from its own features and the features of its neighbours. The graph convolutional network of Kipf and Welling [1] averages neighbour features with fixed weights from the graph structure. The message-passing framework of Gilmer *et al.* [3] generalises this: a *message* is computed on every edge from the states of the two end nodes and the edge features, the messages that arrive at a node are *aggregated* (for example by a sum), and the node state is *updated*. The model in this project follows this pattern. Graph attention networks [2] let the network learn how much each neighbour should contribute, and they will replace the fixed sum in the 100% version.

2.2 Spatio-Temporal Graph Models

Many problems have a graph structure that changes over time. Traffic forecasting is the standard example: the nodes are road sensors and the signals are speed or flow. Yu *et al.* [4] combine graph convolution with temporal convolution to predict traffic. The same idea applies to a race. Here the “road” is the changing neighbourhood of cars around the circuit, and the “speed” is the time lost in a sector.

2.3 Position of this Work

Table 2: Qualitative comparison of modelling choices for traffic delay.

Approach	Uses other cars?	Edge information?	Handles time?
Single-car regression (e.g. gradient boosting)	Only as hand-made features (gap)	No	Via lag features
Single-car sequence model (LSTM)	Only as hand-made features	No	Yes
Graph convolution [1]	Yes	Fixed structure	No
Graph attention [2]	Yes	Learned weights	No
ST-GNN (this project, core)	Yes	Weighted, directed (RBF)	Yes (pooling)
ST-GNN (this project, 100%)	Yes	Weighted, directed + attention	Yes (temporal convolution)

The central idea is that the interaction between two cars is not a feature of one car but a property of the pair. A graph keeps this pair information as an explicit edge. The two baselines of the 100% plan (isolated single-car XGBoost and LSTM) will test whether this explicit structure improves the prediction.

2.4 Tools

The FastF1 library gives access to official timing and telemetry data [8]. PyTorch is used for the model [6], the Adam optimiser for training [5], and scikit-learn for the regression metrics [7].

3 Requirements and Design Rationale

3.1 Requirements

Table 3: Requirements of the core (50%) implementation.

ID	Type	Requirement	Status
R1	Functional	Load real race telemetry and cache it locally	Met
R2	Functional	Align all cars on a common time grid	Met
R3	Functional	Compute the gap to the car ahead at each step	Met
R4	Functional	Build a directed, weighted graph per time step	Met
R5	Functional	Compute clean-air delay targets per mini-sector	Met (see §9)
R6	Functional	Train and evaluate a spatio-temporal GNN	Met
R7	Functional	Visualise the graph and the traffic statistics	Met
R8	Non-functional	Rebuild all graphs in about half a minute from cache	Met (about 25 s)
R9	Non-functional	One command reproduces the full run	Met

3.2 Design Decisions

Table 4: Main design decisions and their rationale.

Decision	Alternative	Rationale
4 Hz common time grid	Native, irregular samples	One grid makes every snapshot a complete graph and keeps the memory small. 0.25 s is fine enough to follow gap changes and coarse enough to run on a laptop.
Directed edges leader→follower	Undirected edges	Wake travels from the leader to the follower. A directed edge keeps the cause and the effect separate and lets the model learn an asymmetric influence.
Edge only to the car directly ahead	Edges to all cars within range	It keeps the graph sparse (about 6 edges per snapshot) and matches the physics: the car directly ahead produces the wake. Longer-range influence is left to multi-hop message passing.
Gaussian RBF weight 2 s edge limit	Binary edge; inverse gap $1/\Delta t$ 1.0 s or 3.0 s	It is smooth, bounded in $(0, 1]$, and has no singularity at small gaps. The width σ has a clear meaning in seconds. Beyond about 2 s the RBF weight is below 0.14. A 3 s limit would match the clean-air definition but would add many weak edges.
Clean-air baseline from the same race	Fixed reference lap	It adapts to track temperature, fuel and tyre state of this race. The price is that the baseline depends on the data.
Masked L1 loss	Mean squared error	L1 is less sensitive to the large outliers in the target (see Figure 7). A mask is needed because not every car has a label in every window.
Chronological split	Random split	A random split would mix neighbouring windows (which overlap in time) between train and test and would overstate performance.
Small model (3,393 parameters)	Large model	The 50% goal is a fast, inspectable baseline. A small model also exposes problems in the data earlier.

3.3 Two Fixed Rules

The project fixes two rules in the specification. They are called “laws” in the project documents.

Law 1 – Edge rule

A directed disturbance edge from the leader j to the follower i exists if and only if $0 < \Delta t_{ij} \leq 2.0$ s.

Law 2 – Clean-air target isolation

The target is the *excess* time over a clean-air baseline, $\Delta t_{\text{sector}} = t_{\text{actual}} - \text{median}(t_{\text{sector}} \mid \Delta t > 3.0 \text{ s})$. The model never receives a clean-air reference as an input feature.

4 Data Ingestion and Preprocessing

4.1 Source Data

The race session of the 2021 Abu Dhabi Grand Prix (58 laps) is loaded with FastF1. For each driver, the per-lap telemetry is concatenated and sorted by time. The channels used are *Speed*,

Throttle and *Distance*. The first call downloads the data. After that, the aligned table is read from a Parquet file, which avoids repeated network access (requirement R1).

4.2 Schema of the Aligned Table

Table 5: Columns of the aligned telemetry table.

Column	Type	Description
TimeSec	float	Session time in seconds, uniform grid with step 0.25
Driver	str	Driver identifier (racing number as a string)
Speed	float	Speed in km/h, linearly interpolated
Throttle	float	Throttle position in percent, interpolated
Distance	float	Cumulative distance in metres, interpolated
LapNumber	int	Derived: $\lfloor d/L \rfloor + 1$
MiniSector	int	Derived: sector index in $\{0, \dots, 19\}$
Compound	str	Constant MEDIUM in the core version (placeholder)
TyreLife	float	Set equal to the lap number in the core version
IsPitLap	bool	Constant False in the core version (placeholder)
DriverAhead	str	Driver directly ahead by distance, or None
DeltaT	float	Time gap to the car ahead in seconds (99 if none)

4.3 Temporal Alignment

The telemetry of each car is sampled at its own irregular time stamps. Every channel is linearly interpolated onto one common grid with $f_s = 4$ Hz ($\Delta\tau = 0.25$ s). The grid starts at the earliest and ends at the latest time stamp found over all cars. The aligned table has 416,879 rows for 19 drivers, that is, 21,941 time steps per driver ($416,879/19 = 21,941$ exactly). At 4 Hz this is about 91.4 minutes of session time.

4.4 Mini-Sectors and Lap Counter

The circuit is divided into $S = 20$ equal mini-sectors. With lap length $L = 5281$ m, each mini-sector is $L/S = 264.05$ m long. For a car at cumulative distance d ,

$$s = \min\left(\left\lfloor \frac{(d \bmod L)}{L/S} \right\rfloor, S - 1\right), \quad \text{lap} = \left\lfloor \frac{d}{L} \right\rfloor + 1. \quad (4)$$

4.5 Gap to the Car Ahead

Algorithm 1 computes the gap at one time step. The cars are sorted by cumulative distance. For the follower i and the car j directly ahead,

$$\Delta t_{ij} = \begin{cases} \frac{d_j - d_i}{\max(v_i/3.6, 10)} & 0 < d_j - d_i < 1500 \text{ m}, \\ 99 & \text{otherwise,} \end{cases} \quad (5)$$

where v_i is the follower speed in km/h. The division by 3.6 converts to m/s, and the floor of 10 m/s prevents very large gaps when a car is almost stationary. A value of 99 s means “no car close enough ahead”.

Algorithm 1 Gap to the car ahead at one time step**Input:** Rows of all cars at time t with Distance d and Speed v **Output:** DriverAhead and DeltaT for every car

```

1: Sort cars by  $d$  in descending order:  $c_1, c_2, \dots, c_N$ 
2: ahead[ $c_1$ ]  $\leftarrow$  None; gap[ $c_1$ ]  $\leftarrow$  99
3: for  $k = 2$  to  $N$  do
4:    $\Delta d \leftarrow d(c_{k-1}) - d(c_k)$ 
5:    $u \leftarrow \max(v(c_k)/3.6, 10)$ 
6:   if  $0 < \Delta d < 1500$  then
7:     gap[ $c_k$ ]  $\leftarrow \Delta d / u$ 
8:   else
9:     gap[ $c_k$ ]  $\leftarrow$  99
10:  end if
11:  ahead[ $c_k$ ]  $\leftarrow c_{k-1}$ 
12: end for

```

4.6 Clean-Air Targets

Algorithm 2 computes the targets. For every (driver, lap, mini-sector) segment, the duration is the difference between the last and the first time stamp inside the segment. A segment is called *clean* when its mean gap is above 3 s. The baseline of a mini-sector is the median duration of the clean segments of that sector (or the global median if there is no clean segment). The target is clipped to $[-1, 5]$ s:

$$\Delta t_{\text{sector}} = \text{clip}(t_{\text{actual}} - t_{\text{baseline}}(s), -1, 5). \quad (6)$$

The run produced 19,768 target segments.

Algorithm 2 Clean-air baseline and delay targets**Input:** Aligned table; clean-air gap $g_c = 3$ s**Output:** Table of segments with DeltaT_Sector

```

1: Group rows by (Driver, LapNumber, MiniSector, Compound)
2: for each group do
3:   Duration  $\leftarrow$  max(TimeSec) – min(TimeSec)
4:   AvgGap  $\leftarrow$  mean(DeltaT)
5: end for
6:  $\mathcal{C} \leftarrow$  segments with AvgGap  $> g_c$  ▷ clean segments
7: for each mini-sector  $s$  do
8:    $b_s \leftarrow$  median{Duration : segment  $\in \mathcal{C}$ , MiniSector =  $s$ }
9: end for
10: DeltaT_Sector  $\leftarrow$  clip(Duration –  $b_s$ , –1, 5)

```

4.7 Data Quality Notes

The review of the code and the output revealed several properties of the data that affect the later analysis. They are recorded here so that they can be tested one by one.

1. **Number of cars.** The aligned table has 19 drivers. To our knowledge, 20 cars started the race. The reason for the missing car has not been checked yet; the loader logs a warning when the telemetry of a driver cannot be read.

2. **Derived lap and sector.** The lap number and the mini-sector are computed from integrated distance and a fixed lap length, not taken from the official lap table. If the integrated distance drifts from the true track position, sector and lap assignments drift with it. This must be compared with the official lap numbers.
3. **Edge behaviour of the interpolation.** The common grid spans the earliest to the latest time stamp of all cars, and linear interpolation holds the end values outside the data range of a car. A car that has finished or retired could therefore remain as a “stationary car” at the end of its data. The effect on the graph near the end of the race has not been measured.
4. **Placeholders.** The tyre compound is constant, and the tyre age equals the lap number. Pit laps are not marked. The core model therefore cannot see tyre state or pit stops.
5. **Quantisation.** Because durations come from 4 Hz samples, a sector duration is a multiple of 0.25 s. A mini-sector of 264 m is driven in roughly 3 to 5 seconds at race speeds (an estimate), so a step of 0.25 s is a few percent of the sector time and limits the smallest delay that can be resolved.

5 Dynamic Graph Construction

5.1 Edge Rule and Weight

A directed edge from leader j to follower i exists at time t if and only if the follower is directly behind the leader and $0 < \Delta t_{ij} \leq 2.0$ s (Law 1). The edge weight is the Gaussian RBF of the gap:

$$e_{ij} = \begin{cases} \exp\left(-\frac{\Delta t_{ij}^2}{2\sigma^2}\right) & 0 < \Delta t_{ij} \leq 2.0 \text{ s}, \\ 0 & \text{otherwise,} \end{cases} \quad \sigma = 1.0 \text{ s.} \quad (7)$$

The weight is close to 1 for a car that is just behind the leader and falls smoothly to $e^{-2} \approx 0.135$ at the 2 s limit (Figure 1).

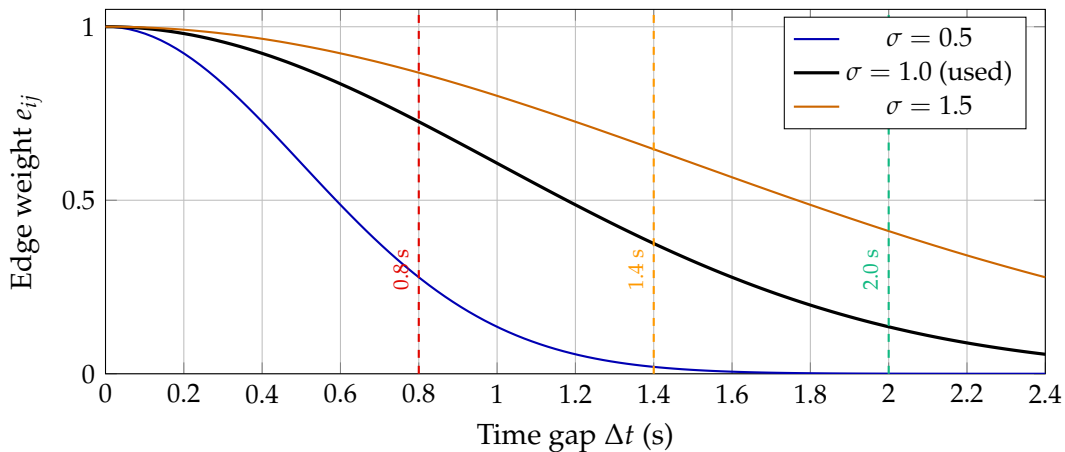


Figure 1: Gaussian RBF edge weight for three kernel widths. The dashed lines mark the tier limits and the edge limit at 2 s.

5.2 Kernel Width Sensitivity

The width σ controls how fast the influence falls with the gap. Table 6 shows the weights for selected gaps (computed from the formula). With $\sigma = 0.5$ s the weight at 1.5 s is almost zero, and the 2 s limit would be meaningless. With $\sigma = 1.5$ s the weight at 2 s is still 0.41, so the cut-off at 2 s would create a visible step. The value $\sigma = 1.0$ s gives a weight of 0.135 at the limit, which is a gentle cut-off. A study of σ is planned in the experiments of Section 12.

Table 6: Edge weight e_{ij} for selected gaps and kernel widths.

Kernel width	$\Delta t = 0.5$ s	$\Delta t = 1.0$ s	$\Delta t = 1.5$ s	$\Delta t = 2.0$ s
$\sigma = 0.5$ s	0.6065	0.1353	0.0111	0.0003
$\sigma = 1.0$ s (used)	0.8825	0.6065	0.3247	0.1353
$\sigma = 1.5$ s	0.9460	0.8007	0.6065	0.4111

5.3 Disturbance Tiers

For display and analysis, edges are grouped in three tiers. The thresholds on e_{ij} follow from the thresholds on the gap, with $\sigma = 1$: $e(0.8) = 0.726$, $e(1.4) = 0.375$, $e(2.0) = 0.135$. The inverse of the kernel, $\Delta t = \sqrt{-2\sigma^2 \ln e}$, recovers the gap from a stored weight, and the visualiser uses this to label the arrows.

Table 7: Disturbance tiers used for graph visualisation.

Tier	Colour	Gap Δt	Weight e_{ij}	Meaning
Severe dirty air	Red	< 0.8 s	≥ 0.726	Strong wake, DRS-train range
Moderate wake	Orange	$0.8\text{--}1.4$ s	$0.375\text{--}0.726$	Noticeable disturbance
Light wake	Green	$1.4\text{--}2.0$ s	$0.135\text{--}0.375$	Minor disturbance
Clean air	—	> 2.0 s	0 (no edge)	No disturbance

Note. The project documents attach indicative downforce-loss percentages to the tiers. These are design assumptions. They have not been measured or validated in this work, and this report does not rely on them.

5.4 Node Features

Each node (car) has four normalised features:

$$\mathbf{x}_i = \left[\frac{\text{Speed}}{320}, \frac{\text{Throttle}}{100}, \frac{\text{TyreLife}}{55}, \frac{\text{MiniSector}}{20} \right]. \quad (8)$$

Normalisation keeps all inputs in a similar range, which makes training stable. Note that the sector index appears as a single scaled number, so the model must learn the (non-linear) effect of the track position itself.

5.5 Construction Algorithm

Algorithm 3 Build the graph snapshot at time t

Input: Rows at time t ; threshold $g_{\max} = 2.0$; width $\sigma = 1.0$; index map π (driver $\rightarrow$ node)

Output: Snapshot ($\mathbf{X}_t, \mathcal{E}_t, \mathbf{e}_t$)

```

1:  $\mathbf{X}_t \leftarrow \mathbf{0} \in \mathbb{R}^{N \times 4}$ ;  $\mathcal{E}_t \leftarrow \emptyset$ 
2: for each row  $r$  at time  $t$  do
3:    $i \leftarrow \pi(r.\text{Driver})$ 
4:    $\mathbf{X}_t[i] \leftarrow [r.\text{Speed}/320, r.\text{Throttle}/100, r.\text{TyreLife}/55, r.\text{MiniSector}/20]$ 
5:    $j \leftarrow \pi(r.\text{DriverAhead})$ ;  $g \leftarrow r.\text{DeltaT}$ 
6:   if  $j$  exists and  $0 < g \leq g_{\max}$  then
7:     add edge ( $j \rightarrow i$ ) with weight  $\exp(-g^2/2\sigma^2)$ 
8:   end if
9: end for
```

5.6 Structural Property: The Graph is a Set of Chains

Because each car has an edge only to the car *directly* ahead, every node has at most one incoming edge and at most one outgoing edge. The graph at any time step is therefore a collection of directed paths (“chains”) plus isolated cars. This has two practical consequences. The adjacency matrix has at most one non-zero entry in each row and each column (visible in Figure 5). And a model that uses a single message-passing layer sees only the immediate leader of each car. The cumulative effect of a long train (the leader’s leader, and so on) can reach a car only through multiple hops or through the temporal dimension.

5.7 Graph Statistics

Table 8: Output of the graph construction stage.

Quantity	Value
Aligned telemetry rows	416,879
Drivers (nodes per graph)	19
Graph snapshots (4 Hz)	21,941
Total directed edges	137,355
Average active edges per snapshot	6.26
Maximum possible edges per snapshot (chain of 19 cars)	18
Mini-sector target segments	19,768

On average, about one third of the possible “follower slots” carry an edge ($6.26/18 \approx 0.35$). The rest of the time, cars run more than 2 s behind the car ahead.

5.8 Worked Example

Consider a follower 0.60 s behind its leader. The weight is $e = \exp(-0.60^2/2) = \exp(-0.18) \approx 0.835$, which is above 0.726, so the edge is red. For a gap of 1.30 s, $e = \exp(-0.845) \approx 0.430$. This is between 0.375 and 0.726, so the edge is orange. The latter value is the one that appears for the pair 31 OC0 $\rightarrow$ 3 RIC in Table 12.

6 Model: CoreSTGNN

6.1 Architecture Overview

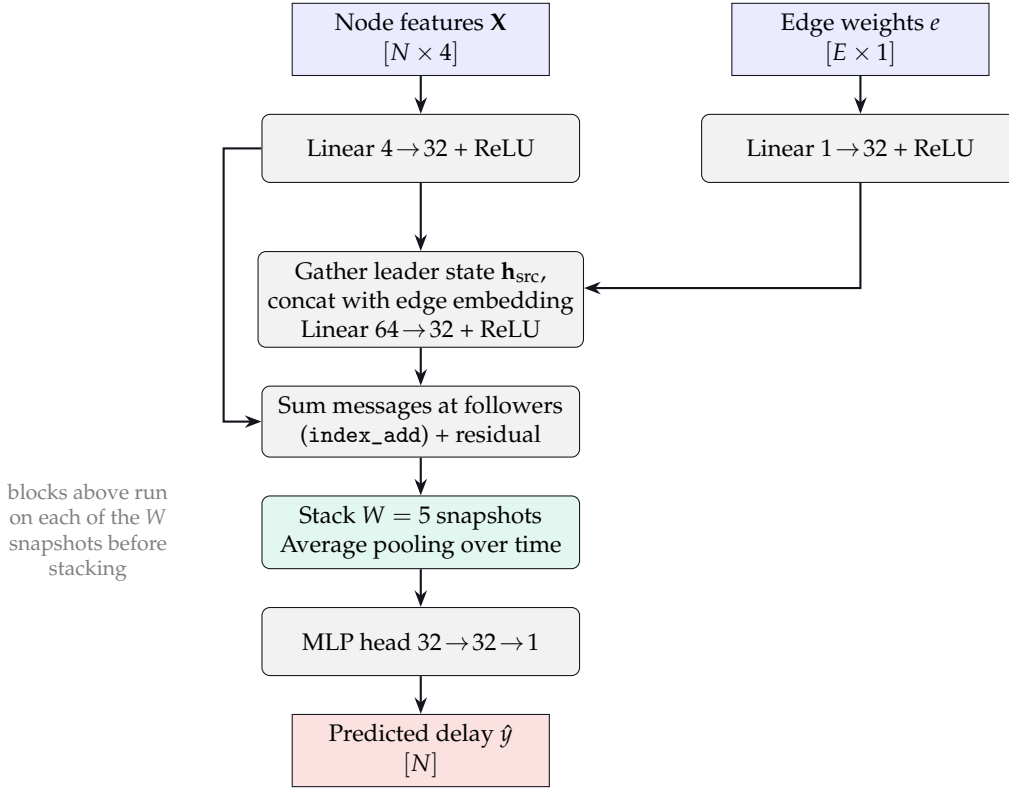


Figure 2: Architecture of CoreSTGNN. The spatial part (projection and message passing) runs on every snapshot of the window. The results are pooled over time and mapped to one delay value per car.

6.2 Spatial Message Passing

For each snapshot, node states are first projected:

$$\mathbf{h}_i^{(0)} = \text{ReLU}(W_p \mathbf{x}_i + b_p), \quad \mathbf{h}_i \in \mathbb{R}^{32}. \quad (9)$$

For every edge $(j \rightarrow i)$, the edge weight is embedded and combined with the leader state to form a message. The messages are summed at the follower, and the sum is added to the follower’s own state (a residual connection):

$$\mathbf{m}_{ji} = \text{ReLU}(W_m [\mathbf{h}_j \parallel \text{ReLU}(W_e e_{ji} + b_e)] + b_m), \quad (10)$$

$$\mathbf{h}'_i = \mathbf{h}_i + \sum_{j \in \mathcal{N}(i)} \mathbf{m}_{ji}. \quad (11)$$

Here $\mathcal{N}(i)$ is the set of leaders of car i . In this graph it has at most one element (Section 5). The sum is still written in general form, because the 100% version may allow more than one leader.

6.3 Temporal Pooling

The states of the $W = 5$ snapshots of a window are stacked and averaged over time:

$$\bar{\mathbf{h}}_i = \frac{1}{W} \sum_{w=1}^W \mathbf{h}_i^{(w)}. \quad (12)$$

The mean treats all snapshots as equally important and ignores their order. It cannot represent a trend (for example, a gap that closes quickly). A temporal convolution is planned to fix this.

6.4 Regression Head

A two-layer perceptron ($32 \rightarrow 32 \rightarrow 1$ with ReLU) maps $\bar{\mathbf{h}}_i$ to the predicted delay $\hat{y}_i$.

6.5 Parameter Count

Table 9: Layers and trainable parameters of CoreSTGNN.

Layer	Mapping	Formula	Parameters
proj	$4 \rightarrow 32$	$4 \cdot 32 + 32$	160
edge_proj	$1 \rightarrow 32$	$1 \cdot 32 + 32$	64
msg_layer	$64 \rightarrow 32$	$64 \cdot 32 + 32$	2,080
head[0]	$32 \rightarrow 32$	$32 \cdot 32 + 32$	1,056
head[2]	$32 \rightarrow 1$	$32 \cdot 1 + 1$	33
Total			3,393

The temporal pooling layer has no parameters.

6.6 Computational Cost

For a sample with W snapshots, N nodes and E edges per snapshot, the cost of the forward pass is $\mathcal{O}(W(NFD + ED^2))$. With $N = 19$, $E \approx 6$, $D = 32$ and $W = 5$, this is a few hundred thousand operations per sample, which explains why one epoch over 4,112 samples finishes in seconds on a CPU. The gap computation of Algorithm 1 is more expensive in the current implementation, because it looks up the rows of the leader and the follower with a boolean filter at every step. Its cost is quadratic in N per time step, which is acceptable for 19 cars and cached after the first run.

6.7 Limits of the Core Design

- **One hop.** One layer sees only the direct leader. A car that is third in a train cannot see the first car.
- **No learned attention.** All edges contribute through their weight embedding only; the model cannot decide that one leader matters more than another.
- **No order in time.** Average pooling ignores the order of the snapshots.
- **Weak inputs.** Four features, a constant compound and no pit-lap information.

These limits are the main reason for the 100% architecture of Section 12.

7 Experimental Setup

7.1 Training Samples

A training sample is a window of $W = 5$ consecutive snapshots paired with the target of the snapshot with index $i + W + K$, where i is the first snapshot of the window and $K = 5$. The last snapshot of the window has index $i + W - 1$, so the target lies $K + 1 = 6$ snapshots (1.5 s) after the last input. Windows start every 4 snapshots (1 s). Figure 3 shows the layout.

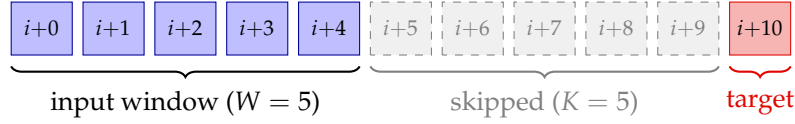


Figure 3: Layout of one training sample. Each box is one 0.25 s snapshot.

The construction yields 5,483 samples; every window had at least two labelled cars. Algorithm 4 summarises the process.

Algorithm 4 Build the window dataset

Input: Snapshots $\mathcal{S}_0, \dots, \mathcal{S}_{T-1}$; target table; $W = 5, K = 5$, stride 4

Output: List of samples (window, target vector $\mathbf{y}$, mask $\mathbf{m}$)

```

1: for  $i = 0, 4, 8, \dots, T - W - K - 1$  do
2:   window  $\leftarrow \mathcal{S}_i, \dots, \mathcal{S}_{i+W-1}$ ; target snapshot  $\leftarrow \mathcal{S}_{i+W+K}$ 
3:   for each driver  $d$  with node index  $n$  do
4:      $s \leftarrow$  mini-sector of  $d$  in the target snapshot
5:     look up the label of  $(d, s)$  in the target table
6:     if a label exists then  $y_n \leftarrow$  label;  $m_n \leftarrow 1$ 
7:     else  $m_n \leftarrow 0$ 
8:     end if
9:   end for
10:  if  $\sum_n m_n \geq 2$  then store the sample
11:  end if
12: end for
```

7.2 Data Split

The samples are split chronologically: the first 75% (4,112 windows, roughly laps 1–43) train the model, and the last 25% (1,371 windows, roughly laps 44–58) test it. The lap ranges are approximate, because the split is by sample index and not by lap boundary.

7.3 Hyper-parameters

Table 10: Hyper-parameters of the core experiment.

Parameter	Value
Input dimension F	4
Hidden dimension D	32
Window length W / offset K	5 / 5 snapshots
Window stride	4 snapshots (1 s)
Edge threshold / kernel width σ	2.0 s / 1.0 s
Optimiser	Adam, learning rate 3×10^{-3} , weight decay 10^{-4}
Loss	Masked L1
Batch size	1 window per optimiser step
Epochs	10 (full run); 3 (quick run in Fig. 4)
Sampling rate	4 Hz
Random seed	Not fixed in the current code

7.4 Loss and Metrics

Not every car has a target in every window, so a mask $m_i \in \{0, 1\}$ is used, and the loss is the masked mean absolute error:

$$\mathcal{L} = \frac{\sum_i m_i |\hat{y}_i - y_i|}{\max(\sum_i m_i, 1)}. \quad (13)$$

Test quality is reported with the mean absolute error and the coefficient of determination, both computed over all labelled (window, car) pairs of the test set:

$$\text{MAE} = \frac{1}{n} \sum_k |\hat{y}_k - y_k|, \quad R^2 = 1 - \frac{\sum_k (y_k - \hat{y}_k)^2}{\sum_k (y_k - \bar{y})^2}. \quad (14)$$

$R^2 = 0$ corresponds to always predicting the mean of the test labels, and $R^2 = 1$ to a perfect prediction. A value of 0.29 therefore means that the model explains about 29% of the variance of the labels beyond a constant prediction.

7.5 Reproducibility and Commands

```
# graph building and visualisation only (about 23 s with cache)
uv run python hammertime_50/demo_core.py --mode graph --snapshot-idx 3000

# full pipeline: graph + training + evaluation + analytics
uv run python hammertime_50/demo_core.py --mode full --epochs 10

# use synthetic telemetry instead of the real race
uv run python hammertime_50/demo_core.py --synthetic --mode full
```

The Parquet cache makes the data stage deterministic after the first download. The training stage is *not* yet fully deterministic, because the random seed is not fixed (this is recorded in the plan).

7.6 Runtime

Table 11: Stage timing taken from the execution log (3-epoch run, cached data).

Stage	Time
Load cached Parquet → clean-air targets ready	about 1.5 s after load
Build 21,941 graph snapshots	about 25 s
Dataset build, 3 epochs, evaluation, analytics	about 80 s (remainder)
Total	113.24 s

8 Results

8.1 Execution Log

Figure 4 shows a complete run of the pipeline with 3 epochs. All five stages finish without error. The only warning is a harmless font-weight message from the plotting library.

```
PS M:\HammerTime> uv run python hamvertime_50/demo_core.py --mode full --epochs 3
=====
HAMMERTIME 50%: 2021 ABU DHABI GRAND PRIX (FASTF1 CORE DEMO)
Mode: FULL | Target Snapshot: 3000 | Epochs: 3
=====

[1/5] Ingesting & Aligning Race Telemetry (FastF1: 2021 Abu Dhabi GP)...
2026-10-06 15:18:28,854 [INFO] Loading cached real FastF1 telemetry from: data\cache\aligned_2021_abu_dhabi_r.parquet
[OK] Successfully loaded actual FastF1 telemetry for 2021 Abu Dhabi Grand Prix!
Drivers on grid: 10, 11, 14, 16, 18, 22, 3, 31, 33, 4, 44, 47, 5, 55, 6, 63, 7, 77, 99
Computing actual clean-air baselines & Delta t sector targets...
2026-10-06 15:18:30,326 [INFO] Calculated targets for 19768 mini-sector segments.
Aligned 416879 telemetry rows. Extracted 19768 sector targets.

[2/5] Constructing Dynamic Spatio-Temporal Graph Snapshots...
2026-10-06 15:18:55,504 [INFO] Built 21941 core graph snapshots for 19 drivers.
[OK] Built 21941 snapshots with 137355 total directed disturbance edges.
Average active disturbance edges per snapshot: 6.26
[OK] Persisted 21941 graph snapshots to disk: data\cache\core_graph_snapshots.pt

[3/5] Visualizing Built Dynamic Graph Snapshot (Red, Orange, Green Tiers)...
2026-10-06 15:18:59,699 [WARNING] findfont: Failed to find font weight heavy for DejaVu Sans, now using 700.
[OK] Rendered multi-tier graph topology & matrix to: M:\HammerTime\hamvertime_50\core_graph_visualization.png
[OK] Copied visual directly to root: M:\HammerTime\core_graph_visualization.png

--- DYNAMIC GRAPH INTERACTION INSPECTION (WHO DISTURBS WHOM) ---
Timestamp: 750.00 s | Nodes: 19 cars | Active Disturbance Edges: 11
-----
Leader Car (j)  ->  Follower Car (i)  Time Gap (Δt)  Weight (eij)  Aerodynamic Tier
-----
16 LEC          ->  22 TSU             0.53 s        0.8679        [SEVERE DIRTY AIR]
99 GIO          ->  10 GAS             0.19 s        0.9815        [SEVERE DIRTY AIR]
22 TSU          ->  77 BOT             1.75 s        0.2158        [LIGHT WAKE]
3 RIC           ->  14 ALO             0.56 s        0.8549        [SEVERE DIRTY AIR]
77 BOT          ->  31 OCO             1.73 s        0.2241        [LIGHT WAKE]
4 NOR           ->  16 LEC             1.92 s        0.1569        [LIGHT WAKE]
31 OCO          ->  3 RIC             1.30 s        0.4300        [MODERATE WAKE]
5 VET           ->  18 STR             1.73 s        0.2252        [LIGHT WAKE]
6 LAT           ->  47 MSC             1.44 s        0.3544        [LIGHT WAKE]
47 MSC          ->  63 RUS             1.32 s        0.4200        [MODERATE WAKE]
18 STR          ->  7 RAI              0.35 s        0.9411        [SEVERE DIRTY AIR]
-----

[4/5] Training Core ST-GNN on Real 2021 Abu Dhabi Grand Prix Targets...
Constructed 5483 real spatio-temporal training samples.
Data Split: Train = 4112 windows (Laps 1-43) | Test = 1371 windows (Laps 44-58)

--- TRAINING CONVERGENCE (REAL DATA) ---
Epoch 01/03 | Real Train L1 Loss: 1.2075 s / sector
Epoch 02/03 | Real Train L1 Loss: 1.1363 s / sector
Epoch 03/03 | Real Train L1 Loss: 1.1290 s / sector

--- TEST EVALUATION ON UNSEEN RACE WINDOWS ---
Test Evaluation on Real Telemetry (Laps 44-58):
>> Real Test MAE: 1.1521 s (Target < 0.150 s) -> NEEDS TUNING (SC Yellows)
>> Real Test R2: 0.2892 (Target 0.68 - 0.82) -> NEEDS TUNING (SC Yellows)

[5/5] Executing Traffic Analytics & Generating Real Analytics Visual...
Detected 9804 DRS-train disturbance events during the Grand Prix.
[OK] Saved real 2021 Abu Dhabi analytics chart to: M:\HammerTime\hamvertime_50\core_analytics.png
[OK] Copied analytics chart directly to root: M:\HammerTime\core_analytics.png

2021 Abu Dhabi core demonstration executed successfully in 113.24 seconds.
=====
```

Figure 4: Terminal output of demo_core.py --mode full --epochs 3.

8.2 Dynamic Graph Snapshot

Snapshot 3000 (time $t = 750$ s, about lap 8) has 11 active edges: 4 red, 2 orange and 5 green. Figure 5 shows the circular topology on the left, with arrows from leader to follower and team colours on the node rings, and the weighted adjacency matrix $A[\text{leader}, \text{follower}]$ on the right.

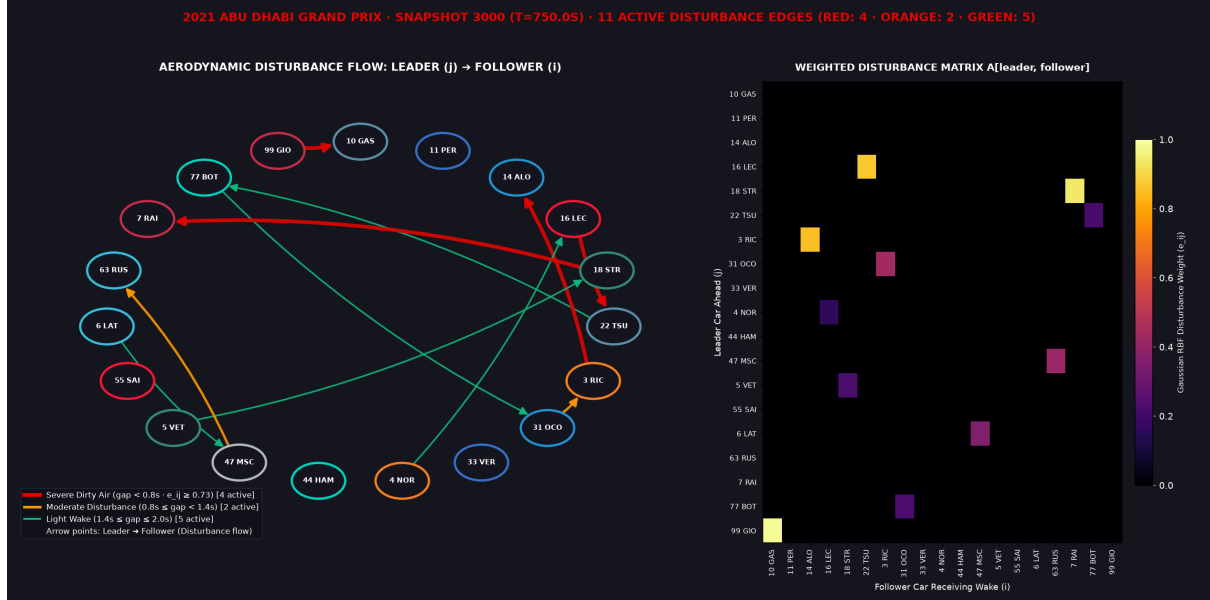


Figure 5: Dynamic disturbance graph at snapshot 3000 ($t = 750$ s).

Table 12: Active edges at snapshot 3000. Gaps are rounded to two decimals in the log.

Leader j	Follower i	Gap (s)	e_{ij}	Tier
99 GIO	10 GAS	0.19	0.9815	Severe
18 STR	7 RAI	0.35	0.9411	Severe
16 LEC	22 TSU	0.53	0.8679	Severe
3 RIC	14 ALO	0.56	0.8549	Severe
31 OCO	3 RIC	1.30	0.4300	Moderate
47 MSC	63 RUS	1.32	0.4200	Moderate
6 LAT	47 MSC	1.44	0.3544	Light
77 BOT	31 OCO	1.73	0.2241	Light
5 VET	18 STR	1.73	0.2252	Light
22 TSU	77 BOT	1.75	0.2158	Light
4 NOR	16 LEC	1.92	0.1569	Light

8.2.1 Reading the Snapshot

The 11 edges link cars into chains. Following the arrows gives

- a chain of seven cars: 4 NOR → 16 LEC → 22 TSU → 77 BOT → 31 OCO → 3 RIC → 14 ALO (gaps 1.92, 0.53, 1.75, 1.73, 1.30, 0.56 s);
- two chains of three cars: 5 VET → 18 STR → 7 RAI, and 6 LAT → 47 MSC → 63 RUS;
- one pair, 99 GIO → 10 GAS;
- four isolated cars: 33 VER, 44 HAM, 11 PER and 55 SAI. They have no car within 2 s ahead.

These are 15 connected cars and 4 isolated ones, which agrees with 11 edges (a chain of n cars has $n - 1$ edges: $6 + 2 + 2 + 1 = 11$). Each leader and each follower appears exactly once in Table 12, which agrees with the at-most-one-in, at-most-one-out property. The tier of every edge agrees with the thresholds of Table 7.

None of the chains contains three consecutive red edges, so a DRS train in the sense of the detector (a chain of at least three cars with every gap ≤ 0.8 s) is not present at this moment. The nearest case is the pair 16 LEC $\rightarrow$ 22 TSU (red), followed by a light edge to 77 BOT.

8.3 DRS-Train Detection

The detector reports 9,804 events over the race. An event is counted at each time step at which a chain of at least three cars with gaps of at most 0.8 s exists. Consecutive steps of the same train are therefore counted separately, and the figure is an upper bound on the number of distinct trains, not a count of trains. A grouping step over consecutive time steps is needed to report distinct trains and their duration (planned in the 100% version, visual V4).

8.4 Training Convergence

Table 13: Training loss (masked L1, seconds per sector).

Epoch	1	2	4	7	10
10-epoch run	1.2110	1.1405	1.1283	1.1110	1.1052

Epoch	1	2	3
3-epoch run (Fig. 4)	1.2075	1.1363	1.1290

The loss decreases monotonically in both runs, which shows that the optimisation works. Most of the improvement happens in the first two epochs; after that the curve is almost flat at about 1.1 s.

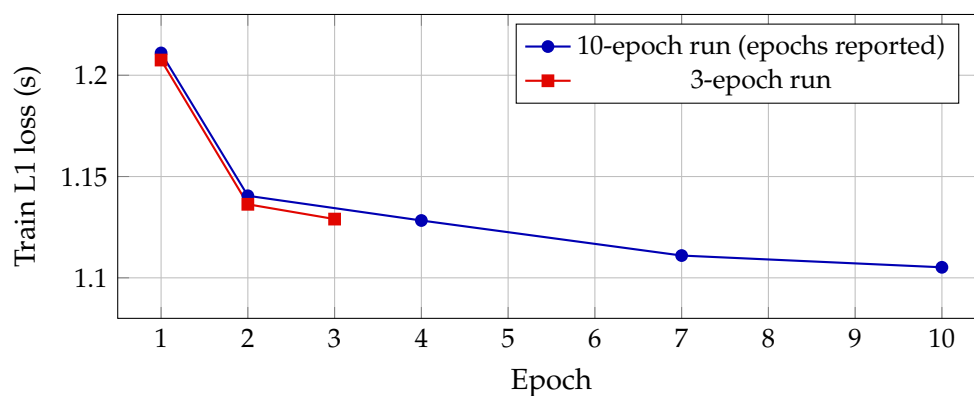


Figure 6: Training loss of the two runs (only the epochs listed in the reports are plotted).

8.5 Test Performance

Table 14: Performance on the held-out closing windows (laps 44–58).

Metric	Target	10 epochs	3 epochs
Test MAE (s)	< 0.150	1.1407	1.1521
Test R^2	0.68–0.82	0.2943	0.2892

8.6 Traffic Analytics

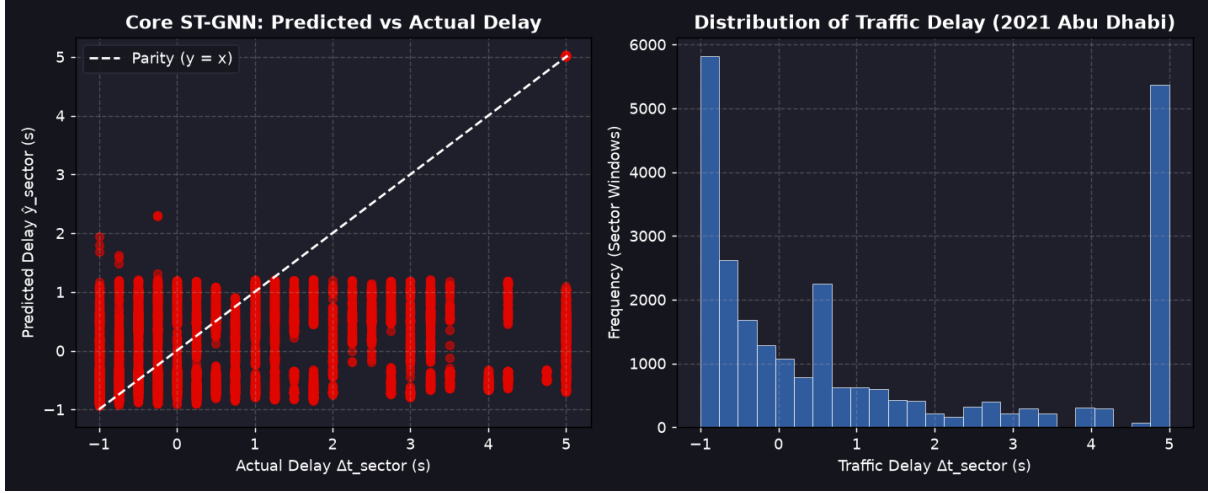


Figure 7: Left: predicted versus actual delay on the test windows (the dashed line is parity). Right: distribution of the actual delay in the test windows.

Both panels of Figure 7 are computed from the test windows only, because the plotting function receives the test labels and predictions. (The project README describes the histogram as covering the whole race. This should be corrected in the next iteration.)

8.6.1 Left Panel: Parity Plot

Points on the dashed line would be perfect predictions. The cloud is instead a set of vertical stripes. All predictions lie between about -1 and 1.2 s whatever the actual value, and the stripe at $+5$ s (the clip limit) is predicted at the same level as the stripe at -1 s. The model has learned roughly a constant with a small spread and does not follow the large delays. The vertical stripes appear because actual values are multiples of 0.25 s (quantisation, Section 4).

8.6.2 Right Panel: Label Distribution

The distribution is bimodal. About 5,800 samples sit at the lowest bin near -1 s (the lower clip limit), and about 5,400 at the upper limit near $+5$ s. A third, small peak appears near 0.5 s. Between these, the frequency falls slowly. So a large part of the labels lie *on a clip limit*. For such samples, the true unclipped value is unknown, and the delay they encode is either very small or very large, not typical.

8.7 Interpretation of the Train–Test Gap

The final training loss (1.105 s) and the test MAE (1.141 s) are close. A close pair of numbers normally indicates under-fitting (the model is too simple, or the input does not contain the information) and not over-fitting. This reading must be used with care for two reasons. The two numbers are computed differently (a mean of per-window means, and a pooled mean), and the label look-up in this version gives the same label for the same (driver, sector) in the training and in the test windows (Section 9).

9 Analysis, Limitations and Threats to Validity

This section lists the causes that may explain the gap between the results and the targets. They are ordered by the evidence available. Two are confirmed by reading the code; the others are hypotheses that the second half will test.

9.1 Findings

1. **Label look-up is not lap-aware (confirmed by code inspection).** The training-set builder creates a dictionary keyed by (driver, mini-sector) and fills it by looping over all target rows. The rows are sorted by driver, then lap. Each assignment overwrites the previous one, so the stored label of a (driver, sector) pair is the value from that driver’s *last lap*. Every sample, in the training and in the test set, then receives that same value for the same (driver, sector), whatever the lap of the target snapshot is (Appendix D). The consequences are:

- the labels do not describe the traffic at the time of the sample, so the model cannot learn the real link between traffic and delay;
- the same label appears in both partitions, so the chronological split does not prevent label overlap between train and test;
- the last lap of the race follows the Safety Car, and its sector times can be very large; this may explain part of the mass at +5 s in Figure 7.

The fix is to key the dictionary by (driver, lap, mini-sector), and to read the lap of the target snapshot.

2. **Sector index from a float product (confirmed by code inspection, minor).** The sector of the target snapshot is recovered as $\text{int}(x * 20.0)$ from a 32-bit float that stores $s/20$. Truncation can map a sector s to $s - 1$ for some values of s . The fix is to store the integer sector separately, or to round.
3. **Safety Car period (diagnosed, not yet quantified).** On lap 53 the Safety Car was deployed after the crash of N. Latifi. Cars ran far below race pace in a compressed queue with no real wake effect. The 50% model has no safety-car mask, and this period lies in the test partition. The 100% version will add safety-car lap flagging. The size of the effect should be measured by recomputing the metrics with these laps removed.
4. **Clipping and outliers (hypothesis).** Clipping to $[-1, 5]$ s creates a large mass at both limits. An L1 loss reduces the pull of outliers, but a model that cannot explain the spikes will predict a central value. Alternatives are to drop non-racing segments (pit, Safety Car, lap 1) instead of clipping them, or to use a robust loss with a wider range.

5. **Baseline definition (hypothesis).** The clean-air median is computed over the whole race. It ignores fuel burn, tyre wear and track evolution, so part of the “delay” is a slow change of pace and not traffic. A baseline per tyre stint or per lap window would reduce this effect.
6. **Single-hop model with weak features (hypothesis).** One message-passing layer, average pooling, four features and no tyre or pit information limit the model.
7. **Derived lap and sector (hypothesis).** The lap and the sector come from integrated distance and a fixed lap length. Drift in the integrated distance would shift sector boundaries. A check against the official lap table is needed.
8. **One race only.** All conclusions come from a single race. Differences between circuits, tyre compounds and weather are not covered. A leave-one-race-out evaluation on several races is the proper test of generalisation.

9.2 Summary Table

Table 15: Issue register with evidence, expected effect and planned action.

#	Issue	Evidence	Expected effect	Planned action
1	Label not lap-aware	Code	High	Key by (driver, lap, sector)
2	Sector from float	Code	Low	Store integer sector
3	Safety Car laps	Race history, plots	High	Flag and mask SC laps
4	Clip spikes	Histogram	Medium	Mask non-racing segments
5	Global baseline	Reasoning	Medium	Per-stint baseline
6	Single hop, few features	Architecture	Medium	GAT, 9 features, temporal conv
7	Derived lap and sector	Code review	Unknown	Compare with lap table
8	Single race	Design	Validity	Multi-race evaluation

9.3 What Works

The infrastructure items are complete and verified by the run: telemetry alignment, gap computation, graph construction (average 6.26 edges per snapshot, in line with a 19-car field), the tiered visualiser, the matrix view, and the training and evaluation loop. The loss decreases, and the visualisation is physically sensible: the closest pairs are red, the cars at the front are isolated, and the chains have the shape that the edge rule implies.

10 Verification and Validation

10.1 Checks Performed

Table 16: Checks that were performed on the 50% implementation.

Check	Method and result	Outcome
Row count consistency	$416,879 = 19 \times 21,941$; snapshots equal time steps	Pass
Average edge count	$137,355/21,941 = 6.26$ matches the reported value	Pass
Edge limit	All logged gaps are in $(0, 2.0]$ s	Pass
Weight/tier consistency	Each of the 11 edges at snapshot 3000 has the tier that its weight implies	Pass
Chain structure	Each leader and follower appears once; $\sum(n_c - 1) = 11$	Pass
Loss decreases	Monotone decrease in both runs	Pass
End-to-end run	All five stages complete; 113.24 s	Pass
Accuracy targets	MAE and R^2 against R8 and R9	Fail

10.2 Checks Planned

- A unit test that rebuilds the label table with the lap key and confirms that labels differ between laps.
- A clean-air test: for cars with no edge, the predicted delay must be close to zero (target ≤ 0.05 s).
- A DRS-train test: the cumulative penalty over a train should lie in the expected envelope (0.40–0.80 s per lap in the project target).
- Fixed random seeds and a repeated run to measure the run-to-run variation of MAE and R^2 .
- A comparison of the derived lap and sector with the official lap table.

11 Project Management

11.1 Status at the Half-Way Point

Table 17: Status of project components at the half-way point.

Component	Status	Approx. progress
FastF1 ingestion, 4 Hz alignment, caching	Complete	100%
Gap computation and clean-air targets	Complete (needs refinement)	80%
Dynamic graph builder (RBF edges)	Complete	100%
Graph visualiser and adjacency matrix	Complete	100%
Core ST-GNN and training loop	Complete (baseline)	70%
Label construction for training	Defect found	40%
Accuracy targets (MAE, R^2)	Not met	20%
Safety car and pit-lap handling	Not started	0%
Attention layers, temporal convolution	Not started	0%
Baseline comparison (XGBoost, LSTM)	Not started	0%
Physics validation suite	Not started	0%
Dashboard and broadcast cards	Not started	0%

11.2 Risk Register

Table 18: Main risks and mitigations.

Risk	Likelihood	Impact	Mitigation
Accuracy targets are not reachable on one race	Medium	High	Add more races; report honest metrics and the gap to target
Label quality limits any model	High	High	Fix look-up first; audit labels against lap table
Safety Car and pit laps dominate errors	High	Medium	Flag and mask; report metrics with and without
FastF1 service or schema change	Low	Medium	Keep the Parquet cache in the repository store
Run-time grows with more races	Medium	Low	Cache graphs; vectorise the gap computation
Over-engineering before baselines exist	Medium	Medium	Build the XGBoost and LSTM baselines early

12 Plan for the Second Half

12.1 From the 50% Core to the 100% System

Table 19: Planned upgrades from the 50% core to the 100% system.

Subsystem	50% core (now)	100% target
Label quality	(driver, sector) look-up; clip $[-1, 5]$	Lap-aware keys; masking of non-racing segments
Race control	None	Safety-car and pit-lap masking
Spatial layer	Additive message passing	2-layer GAT, 4 heads, edge feature, dropout 0.1
Temporal layer	Average pooling, $W = 5$	1-D convolution (kernel 3), $W = 10$
Node features	4 normalised values	9 values: speed, throttle, compound (5-way one-hot), tyre age, mini-sector
Baselines	None	Single-car XGBoost and LSTM on the same split
Validation	DRS count only	Clean-air test; DRS-train test
Presentation	Static PNG files	Streamlit dashboard; broadcast cards

12.2 Planned Attention Layer

The 100% model replaces the fixed sum with attention over the leaders of a car. For a follower i and a leader $j \in \mathcal{N}(i)$, the attention coefficient has the form

$$\alpha_{ji} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [W\mathbf{h}_i \parallel W\mathbf{h}_j \parallel W_e e_{ji}]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [W\mathbf{h}_i \parallel W\mathbf{h}_k \parallel W_e e_{ki}]))}, \quad \mathbf{h}'_i = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ji} W\mathbf{h}_j\right), \quad (15)$$

as in graph attention networks [2], with the edge weight e_{ji} as an additional input. With two stacked layers the receptive field grows to two hops, so a car can see the leader of its leader.

12.3 Experiment Plan

Each change is tested alone against the corrected baseline, so that its contribution can be measured.

Table 20: Planned experiments.

ID	Change	Question answered
E0	Lap-aware labels (core model, nothing else)	What is the true baseline performance?
E1	Mask Safety Car and pit laps	How much error comes from non-racing laps?
E2	Replace clipping by masking; try Huber loss	Do the clip spikes limit the fit?
E3	Per-stint clean-air baseline	Is part of the delay a pace drift?
E4	9 features with tyre compound and age	Does tyre state help?
E5	Two-layer GAT	Does attention and two hops help?
E6	Temporal convolution, $W = 10$	Does the order of snapshots help?
E7	$\sigma \in \{0.5, 1.0, 1.5\}$, limit $\in \{1.5, 2.0, 3.0\}$ s	How sensitive is the model to the edge definition?
E8	Isolated XGBoost and LSTM baselines	Does the graph structure add value?
E9	Additional races, leave-one-race-out	Does the model generalise?

12.4 Success Criteria

The final system will be accepted if, on clean-racing test windows, $MAE < 0.150$ s and R^2 lies between 0.68 and 0.82, if it beats the single-car baselines on the same split, and if the clean-air and DRS-train physics tests pass. If the targets are not reached, the final report will state the reached values and the evidence for the remaining gap.

12.5 Planned Visual Suite

The 100% system adds nine visuals (V1–V9): a track traffic map, a parity scatter, a cars-by-sectors congestion heat-map, a DRS-train timeline, a baseline comparison, physics validation cards, a “who is slowing you down” drill-down, residual diagnostics, and a clean-air calibration plot. A Streamlit dashboard and ten broadcast cards are planned on top of these.

13 Conclusion

At the half-way point, the project has a working end-to-end pipeline on real Formula 1 telemetry: 416,879 aligned rows, 21,941 dynamic graph snapshots, a physically motivated Gaussian edge weight, a trainable ST-GNN of 3,393 parameters, and a broadcast-style disturbance visualiser. The graph construction behaves as designed: edges follow the 2 s rule, tiers match the weights, and the graph at each time step is a set of chains.

The model does not yet reach the accuracy targets (MAE 1.14 s, R^2 0.29). The critical review found that the main limit is the quality of the training labels: the look-up ignores the lap, so the current numbers are a preliminary baseline. The Safety Car period, target clipping, a global clean-air baseline and a simple single-hop model are further causes. The second half will correct the labels first, then add race-control masking, richer features, attention and temporal convolution, build the baselines, and run the physics validation. Each step will be measured separately, so that the final claims rest on evidence.

References

- [1] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2017.
- [2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. ICLR*, 2018.
- [3] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proc. ICML*, 2017.
- [4] B. Yu, H. Yin, and Z. Zhu, “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,” in *Proc. IJCAI*, 2018.
- [5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [6] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in NeurIPS*, 2019.
- [7] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [8] FastF1 contributors, “FastF1: Python package for accessing Formula 1 timing and telemetry data,” <https://github.com/the0ehrly/Fast-F1>.

A Glossary

Clean air	Running with no car close ahead, so that the airflow over the car is undisturbed.
Dirty air	The turbulent, slower air behind a car. It reduces the downforce of a follower.
DRS	Drag Reduction System; a rear-wing flap that a car within one second of the car ahead may open in marked zones.
DRS train	A group of cars, one behind the other, each within DRS range of the car in front.
Gap	The time difference between two consecutive cars at one place on the track.
Mini-sector	One of 20 equal-length parts of the lap (about 264 m here).
Safety Car	A car that leads the field at reduced speed after an incident; overtaking is not allowed.
Snapshot	The state of the whole field (a graph) at one 0.25 s time step.
Stint	The period between two pit stops, on one set of tyres.

B Repository Layout

Table 21: Files of the 50% core implementation.

File	Purpose
configs/core.yaml	Session, graph and training parameters
src/core_pipeline.py	FastF1 loading, alignment, gaps, clean-air targets, synthetic fallback
src/core_graph.py	Snapshot builder with RBF edge weights
src/core_model.py	CoreSTGNN model
src/core_analytics.py	DRS-train detection, graph and parity plots, driver table
demo_core.py	Command-line runner (full, graph, train)
requirements.txt	Python dependencies

Command-line options of `demo_core.py`

Option	Default	Effect
--mode	full	graph: ingest, build, inspect, visualise; full: all five stages
--snapshot-idx	3000	Snapshot to visualise (about $t = 750$ s, lap 8)
--epochs	10	Number of training epochs
--no-save-graph	off	Do not persist the snapshots to data/cache
--synthetic	off	Use synthetic telemetry instead of FastF1

C Notes on Configuration

The file `configs/core.yaml` already lists parameters of the planned 100% system (9 input features, 2 attention heads, window of 10, safety-car flag, batch size 32, learning rate 0.002, 25 epochs). The 50% code does not read them: `demo_core.py` uses its own constants (4 input features, $W = 5$, $K = 5$, batch size 1, learning rate 0.003, 10 epochs). This is intended for the core stage, and the values will be unified when the 100% model is built.

D Key Code Excerpts

Gap to the car ahead (`core_pipeline.py`)

```
dist_diff = lead_row["Distance"] - fol_row["Distance"]
fol_speed_ms = max(fol_row["Speed"] / 3.6, 10.0)
if 0 < dist_diff < 1500.0:
    dt_gap = dist_diff / fol_speed_ms
else:
    dt_gap = 99.0
```

Edge weight (`core_graph.py`)

```
ahead = row.get("DriverAhead")
gap = float(row.get("DeltaT", 99.0))
if ahead in drv_to_idx and 0.0 < gap <= edge_threshold:
    leader_idx = drv_to_idx[ ahead ]
    w = np.exp(- (gap ** 2) / (2.0 * (sigma ** 2)))
```

```
src.append(leader_idx); dst.append(idx); weights.append(w)
```

Message passing (core_model.py)

```
e_emb = torch.relu(self.edge_proj(edge_attr))
msg = torch.relu(self.msg_layer(torch.cat([h[src], e_emb], dim=-1)))
agg = torch.zeros_like(h)
agg.index_add_(0, dst, msg)
h = h + agg
```

Label look-up (demo_core.py) – see Section 9, item 1

```
target_lookup = {}
for _, row in df_targets.iterrows():
    key = (str(row["Driver"]), int(row["MiniSector"])) # no lap in the key
    target_lookup[key] = float(row["DeltaT_Sector"]) # last lap overwrites

# later, for the target snapshot:
sec_idx = int(target_snap.x[d_idx, 3].item() * 20.0) # float truncation
target_val = target_lookup.get((str(d_name), sec_idx))
```

Suggested corrected look-up

```
# store the lap and the integer sector in each snapshot, then:
target_lookup[(str(row["Driver"]),
               int(row["LapNumber"]),
               int(row["MiniSector"]))] = float(row["DeltaT_Sector"])
```